TempSched: A Temperature-Aware Storage Scheduler for Time Series Across Cloud-Edge-Device

Shuangshuang Cui, Hongzhi Wang✉, Xianglong Liu, Xiaoou Ding
Faculty of Computing, Harbin Institute of Technology, Harbin, China
cuishuangs@stu.hit.edu.cn, wangzh@hit.edu.cn, 23S003029@stu.hit.edu.cn, dingxiaoou@hit.edu.cn

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00076

***Abstract*—Storage scheduling is crucial for time series storage. However, designing an efficient hot and cold tiered storage scheduling strategy for time series across Cloud-Edge-Device (CED) architecture remains challenging. Although numerous research have studied hot and cold classification for relational data, these methods are not suitable for time series which has strong timeliness and complex access patterns. Therefore, in this paper, we present `TempSched`, a temperature-aware storage scheduler for time series across CED, which can identify hot and cold time series and predict data temperature efficiently to perform storage scheduling in advance. By employing Newton's law of cooling and the thermal radiation law, `TempSched` establishs a temperature model and encapsulates data temperature. It supports classifying hot and cold data and scheduling time series across CED. Subsequently, `TempSched` designs a workload prediction model and a frequent timestamp discovery algorithm to forecast access patterns and predict the future temperature. This can timely adjust to hot and cold storage. We validate `TempSched` on a public dataset, and the experimental results show that it can achieve about 94% hit rate for data access on the edge and device, which is 12% better than existing methods. It can help CED avoid storage overhead caused by storing the full data at all three sides, and greatly reduce data transfer overhead.**

***Index Terms*—Time series, Cloud-Edge-Device, Cold data and hot data, Hierarchical storage, Workload forecasting.**

I. Introduction

Storage scheduling is crucial for time series storage. This is motivated by the characteristics of time series and their applications [1]–[3], [6]. Firstly, time series is a series of numerical data points collected over time. Each data point is gathered at discrete time intervals and associated with a timestamp that records when it has been written [5], [9]. It has different **"temperature"** to present their different written time and access frequency. The hot data, whose **"temperature"** is high, indicates frequent or recent accessed. Since the value of time series typically decreases over time, storage scheduling should oversee time series lifecycle and ensure to store important data on high performance media during its most valuable periods. Secondly, as the scale of time series in the Industrial Internet of Things (IIoT) is increasing [7], the Cloud-Edge-Device (CED) collaborative architecture, comprising center cloud, edge servers, and sensor terminals, has replaced the traditional central cloud architecture [4], [8]. It has become a new framework for time series management and analysis, such as intelligent transportation [10], smart factory [11], etc. But storing all data in the cloud is impractical, as it would incur substantial network transmission costs during edge-device task execution. Specially, storage resources within the CED architecture are heterogeneous, especially at the edge, where resources are limited and incapable of hosting complete data replicas. So, storage scheduling can allocate time series storage across different levels of CED architecture based on task characteristics to ensure efficient system operation.

Designing an efficient cold-hot tiered storage scheduler is still challenging for time series across CED.

Fine-grained classification of time series. The access patterns of time series are often dynamic. They can be influenced by seasonal variations, peak business periods, and changed in user behavior. It is difficult to predict accurately which data will become hot data. So quantitatively representing time series temperature is essential to classify cold and hot data [12]. It is crucial for maintaining high operational efficiency and resource utilization across diverse scenarios.

Advance storage scheduling of time series. The time series scale can reach the petabyte (PB) level [10], [11]. It often shows significant fluctuations in workload, such as during peak traffic periods or unexpected events. Storage scheduling in advance according to the historical workload can help the system manage these fluctuations and improve its flexibility. It also ensures that hot data are quickly accessed when needed, which reduces access latency.

Support real-time decision making. Edge computing and devices are often involved in essential tasks such as monitoring and alerting, which require prompt and accurate responses. In this architecture, storage scheduling must effectively support real-time decision making. This capability ensures that critical data are quickly accessible, allowing systems to respond efficiently to events and maintain operational effectiveness [13].

As a result, a storage scheduler that can classify hot and cold time series across CED at a fine granularity and schedule them in advance is indeed.

The current works have studied classifying the hot and cold data in different contexts. For example, the cache replacement strategies represented by Least Recently Used (LRU) [14] and Least Frequently Used (LFU) [15] use timeliness (the most recently accessed data are hot data) and access frequency (the data with the highest frequency in historical data are hot data). Specifically, one way is to consider the timeliness and frequency of data access to classify the hot and cold data [16]. Multiple independent hash functions are adopted in [17] to reduce the chance of false identification of hot data and to provide predictable and excellent performance for hot data identification. The authors in [18] propose a novel hot data identification scheme adopting multiple bloom filters to

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00076

efficiently capture finer-grained recency as well as frequency. Siberia predicts the likelihood of data being accessed in future by exponential smoothing to maximize the main memory hit rate [19]. Gorilla stores the temporal data which collected from the database in the last 26 hours on the hot storage media [23].

Despite their excellent results and wide applications, they are primarily limited to classify the cold and hot relational data but not time series for the following three reasons. First of all, compared to relational data, time series have *temporal sensitivity*, which is defined as follows. Let T denote time, and $D(T)$ denote the time series characteristics (such as access frequency, importance, etc.) at time T. Then, *temporal sensitivity* $TS = \frac{\partial D(T)}{\partial T}$, which indicates the change rate of data characteristics D with respect to time T. For example, real-time weather data are essential for the same day forecasting, while data from a few days ago may have less significance. Existing methods primarily consider access time and frequency, failing to fully capture the data *temporal sensitivity*. Secondly, time series is massive and has different access patterns. These patterns change across various application scenarios [1], [5], [7]. Current relational data hierarchical storage methods do not consider future workloads [20]–[22]. They cannot predict workload changes to adjust the classification of hot and cold data dynamically. The last but not least, the access patterns of time series are periodicity. Although time series has not been accessed for a long time, it still may become important in the future. Most existing methods fail to capture this characteristic, resulting in classifying important data as cold data.

For the equally important hierarchical storage field of cold and hot time series storage, similar works on classify hot and cold time series and focus on future access patterns are in demand. To describe the temperature of the data quantitatively, [20]–[22] modeled the temperature of data based on Newton's law of cooling, in which the increased temperature only depends on the number of access. It fails to describe the relationship between increased temperature and access interval. For instance, the temperature model proposed in [22] is shown in Eq.(1).

$$T(t_n) = T(t_{n-1})e^{-\alpha(t_n - t_{n-1})} + T_{heat} \tag{1}$$

where $T(t_n)$ is the current temperature. $T(t_{n-1})$ is the temperature when data were accessed last time. α is the attenuation coefficient. T_{heat} is the increased temperature.

Between 1:00 and 5:00, we consider two situations: ① data were accessed at 2:00 and 5:00. ② data were accessed at 4:00 and 5:00. The increased temperature at 5:00 in Equation 1 are the same because of the same number of access between ① and ②. However, as accessing to time series is periodic, the probability of data in ② being accessed at the next time point (e.g. 6:00) is higher than in ①. Consequently, the increased temperature in ② should be higher than in ①. Thus, this temperature model still has limitations in performance improvements when used for time series storage as it ignores different access patterns and fails to describe the relationship between increased temperature and access interval. Besides, they cannot take future access patterns into guide tiered storage. Therefore, in this paper, our goal is to design a temperature-aware storage scheduler which can classify hot and cold data according to different access patterns of time series and schedule it in advance. To achieve this goal, there are still three challenges to be addressed.

(C1) How to model the relationship between access intervals and increased data temperature?

(C2) How to predict data temperature and adjust the storage strategy in advance?

(C3) How to exploit temperature-aware storage scheduler to improve query performance?

To address the above challenges, we present `TempSched`, a Temperature-aware storage Scheduler for time series across CED, which can identify hot and cold time series and predict data temperature efficiently to storage schedule in advance. To the best of our knowledge, this is the first work that utilizes temperature-aware storage scheduler for time series across CED. First, to quantitatively describe data temperature and characterize the relationship between increased temperature and access intervals, we model data temperature based on Newton's cooling law and thermal radiation law **(for C1)**. Second, to predict data temperature, we establish a query arrival rate prediction model and propose a frequent access timestamp mining algorithm **(for C2)**. Third, we use `TempSched` to select the optimal storage location across CED, thereby improving data access hit rate and accelerating queries **(for C3)**.

The contributions of this paper are as follows.

(1) We present `TempSched`, which automatically adapts the storage strategy for time series across CED according to the data temperature.

(2) We present time series data temperature calculator using Newton's cooling law and thermal radiation law. It can describe the relationship between increased data temperature and access interval to divide hot and cold time series accurately.

(3) We present time series temperature predictor, which based on query log. It can adjust data in advance.

(4) We evaluate `TempSched` on large scale real world datasets, and the results demonstrate a 12% improvement in hot data access rate compared to other hierarchical storage methods.

The rest of the paper is organized as follows: Section II introduces the related work. Section III analyzes the challenges. Section IV describes the workflow of the proposed approach. Section V conducts experiments and analyzes the experimental results. Section VI concludes and discusses future work.

II. Related Work

We classify the related work into the CED time series storage, hot and cold data hierarchical storage, and workload prediction to compare `TempSched`.

CED Time Series Storage. Currently, the storage strategies for cloud-edge-end time series data are mainly divided into two categories: the centralized ones [24]–[26] and the distributed ones. The distributed-based method [27] can achieve better workload balancing than the centralized methods, but it is difficult to guarantee the consistency of the metadata due to the occurrence of link and node failures. There are already many database products supporting CED systems, but the current storage methods are mainly based on single cloud-side, edge-side, or device-side storage. For example, OpenTSDB [28] only supports cloud-side data storage. It is worth noting that

although Apache IoTDB [29] and TDengine [30] have the ability to deal with the storage requirements of cloud-side and device-side data at the same time, they cannot meet the demand of collaborative processing. This motivates us to seek a storage scheduling method to achieve collaborative hierarchical time series storage across CED.

Hot and Cold Data Hierarchical Storage. The first step towards hierarchical storage of hot-cold data are to find an effective way to distinguish the hot-cold data. LRU [14] and LFU [15], the representation of the classical cache replacement strategies, identify hot and cold data from the perspectives of timeliness and data access frequency to maximize the main memory hit rate. Leveraging Newton's law of cooling as a tool to quantitatively describe two behaviors: data temperature decays with time and data temperature rises with access, [20], [21] explicitly build a data temperature model. In terms of a tiered storage strategy, [20] proposed a temperature model-based cache replacement strategy to realize hot-cold data migration. Faced with the need for hot-cold data identification and hierarchical storage on a temporal database, [22] proposed AdjustDTM which can identify the hot and cold by assigning the attribute "temperature" to the data with adjustable parameters. Gorilla [23] takes the last 26 hours of temporal data collected from monitor devices as hot data. However, the above methods fail to divide the hot and cold time series data in a fine granularity, and they can not schedule it in advance.

Workload Forecasting. For workload prediction tasks, [31] proposed to identify and forecast significant workload migrations depending on SQL-based workloads monitoring with Markov chains, neural networks, and fuzzy logic. QueryBot5000 [32] used an online dynamic clustering approach to process SQL queries, and employs prediction models for each cluster. [33] put forward an idea about using an RNN to predict future workloads of DaaS, while [34] leveraged an online, multi-step workload prediction method based on the auto-encoder framework to forecast the resource utilization and query arrival rate of DBMS. Although these methods can predict workload to get the time series and related fields which will be accessed next, they are unable to predict the timestamp range which accessed frequently. In addition, it is vital to predict the timestamp range of frequently accessed data for hierarchical storage scheduling of hot and cold data.

III. MOTIVATIONS

Existing cold and hot classification methods have some shortcomings, making it unable to store scheduling time series across CED. First of all, most of the existing methods only store data hierarchically based on data access time and frequency. They can not consider the timeliness and different access patterns of time series. Therefore, the use of Newton's law of cooling combined with the thermal radiation law in `TempSched` can achieve a 12% increase in hit rate over the current methods. The second point is that existing methods do not consider the impact of future access patterns on the time series temperature, while `TempSched` can schedule time series in advance by predicting the workload. The third point is that the time series scale can reach PB level and its value changes over time and application scenarios [1], [5], [7], [8], [10], [11], but existing methods cannot dynamically adjust the hot and cold classification of data. Besides, to the best of our knowledge, this is the first work that utilizes a temperature-aware storage scheduler for time series across CED.

`TempSched` employs a temperature calculator and a temperature predictor to guide storage scheduling time of series across CED. By doing so, we not only identify and dynamically adjust the hot and cold time series but also account for the timeliness and different access patterns. It addresses the limitations of existing hot-cold data classification methods mentioned in Section I. Our work focuses on designing precise and dynamic methods for hot-cold classification of time series, which is central to storage scheduling in CED architecture. The temperature calculator establishes a temperature model using Newton's law of cooling and the thermal radiation law. It also encapsulates data temperature to achieve precise hot-cold data classification. To adjust the hot and cold storage of time series in advance, the temperature predictor designs workload prediction models and a frequent timestamp discovery algorithm to predict access patterns. It can forecast the future temperature of time series. This guides the hot-cold storage scheduling and further enhances system performance.

IV. TEMPSCHED APPROACH

In this section, we introduce the workflow of `TempSched`. Afterward, the details of two key modules are described, including *Time Series Temperature Calculator* and *Time Series Temperature Predictor* in Section IV-B and IV-C, respectively. Figure 1 demonstrates the architecture of `TempSched`.

A. Workflow

The workflow of `TempSched` is shown in Figure 1, which is divided into two main independent steps. The first step is to calculate time series temperature. For better hierarchical storage of time series, the challenge is modeling the relationship between access intervals and increased data temperature to classify hot and cold time series. Given time series and workload in CED, `TempSched` firstly uses the temperature calculator to calculate the temperature for time series collected by the device. Then it follows the temperature and CED's three levels storage space to store time series across CED. This module will be detailed in Section IV-B.

The second step is to use a temperature predictor to predict the time series temperature in the future according to historical workload. It is important to note that the temperature predictor will not be used when there is no historical data available. In this case, the temperature calculator can directly compute the data temperature for scheduling. Once enough historical data are accumulated, the temperature predictor will activate to work better with the temperature calculation module for proactive scheduling. The challenge to achieve efficient hierarchical storage is adjusting the storage strategy in advance based on the future temperature, which requires the future workloads. `TempSched` extracts query templates from the logs, clusters query templates, and establishes the query arrival rate prediction model. It also employs the frequent access timestamp mining algorithm to identify frequently accessed time series data and predict the time series temperature in

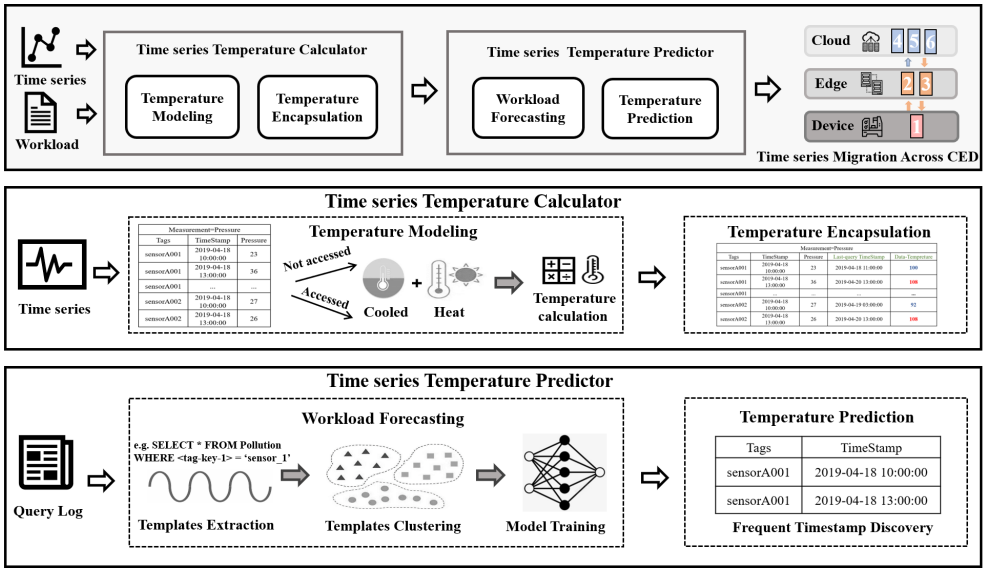


Fig. 1. The Architecture of TempSched.

future. Then it can follow the predicted temperature results as well as CED's three levels storage space to migrate time series in advance. This module will be described in Section IV-C.

B. Time Series Temperature Calculator

In TempSched, we propose a temperature calculator composed of temperature modeling and encapsulation. Calculating and encapsulating the time series temperature using the temperature model can aid in storage scheduling. Therefore, the temperature calculator is the foundation for storage scheduling time series across CED. Due to the different access patterns of time series, it is difficult to establish a temperature model and storage method. The temperature model must quantitatively describe the decreasing temperature of time series over time, as well as the increasing temperature under different access patterns. In addition, the storage method must be capable of handling temperature updates during time series writes.

Key idea. To address the challenges in temperature calculation, the time series temperature calculator establishes a temperature model based on Newton's law of cooling and thermal radiation law. This approach overcomes the limitations of existing methods for identifying hot and cold data. By encapsulating and storing data temperature, it enables on-demand updates of data temperature, ensuring precise and dynamic data management.

Temperature Model. Newton's law of cooling and thermal radiation law are fundamental principles in physics.

Definition 1 (Newton's Law of Cooling): When there is a temperature difference between the surface of an object and its surroundings, the amount of heat dissipated per unit area per unit time is proportional to the temperature difference. The proportionality constant is referred to as the heat transfer coefficient. Newton's Law of Cooling is Eq.(2).

$$\frac{\mathrm{d}T(t)}{\mathrm{d}t} = -a(T(t) - H) \tag{2}$$

where H is the ambient temperature, $T(t)$ is the current temperature of the object, and a is the ratio coefficient of the temperature change rate to the temperature difference.

Definition 2 (Thermal Radiation Law): When a heat source with a radius r transfers heat outward, the heat received by an object located at a distance x can be expressed in Eq.(3).

$$f(x, T_{heat}) = \sigma \frac{{T_{heat}}^4 r^2}{x^2} \tag{3}$$

where T_{heat} is the temperature of heat source, and σ is the thermal radiation coefficient, named Stefan-Boltzmann constant.

In temperature calculator, Newton's law of cooling and thermal radiation law complement each other, the former simulates the natural decay of time series temperature over time, while the latter dynamically adjusts time series temperature based on access frequency and intensity. This combination provides a more comprehensive and accurate temperature model with the following three main advantages.

Quantitative and simplified time series models. Newton's law of cooling describes the process of an object temperature gradually approaching the ambient temperature. This is analogous to data becoming less important when not accessed. The thermal radiation law describes an object radiates energy based on its temperature, offering a precise model to quantify the "heat" of data. Both laws are a simple mathematical model and make it suitable for time series management in CED.

Adaptive to different access patterns of time series. The thermal radiation law can flexibly adjust time series temperature according to different access patterns, enabling comprehensively represent the importance and "heat" of data. Thus it is suitable for different access patterns of time series.

Dynamically adjust time series storage strategies. By accurately measuring time series temperature using two laws, it is possible to dynamically adjust the classification of hot and cold data and storage strategies. This optimizes the utilization

efficiency of storage resources, thereby improving the overall performance and responsiveness of the system.

Next, we describe the temperature model for time series in detail. We use Newton's law of cooling to represent the decay part of the temperature model. Additionally, we propose a concept of a "heat source" to depict the heating process resulting from data access. This "heat source" is equivalent to the data's temperature being raised when accessed.

Definition 3 (Data Temperature Model): The $data_i$ is the i_{th} data point of time series. When $data_i$ is accessed s times in $(t_n - t_{n-1})$, the $T_i(t_n)$ of $data_i$ is defined as:

$$T_i(t_n) = T(t_{n-1})e^{-k(t_n - t_{n-1})} + \gamma s \frac{{T_{heat}}^4}{(t_n - t_{n-1})} \quad (4)$$

where T_{heat} is temperature of the heat source, k denotes the cooling rate and γ denotes the heating rate.

In the following, Eq.(5)-(14) gives the modeling process for the data temperature model.

$$T(t) = (T_0 - H)e^{-kt} + H \quad (5)$$

Integrating Eq.(2), we obtain Eq.(5). Eq.(5) models the $T_f(t_n)$ and neglect the effect of ambient temperature. we obtain Eq.(6), where k denotes the cooling rate.

$$T_f(t_n) = T(t_{n-1})e^{-k(t_n - t_{n-1})} \quad (6)$$

The heat radiation transfers heat as defined by Eq.(7).

$$f(x, T_{heat})\mathrm{d}s\mathrm{d}t = \mathrm{d}Q = C\mathrm{d}m\mathrm{d}T_r \quad (7)$$

The actual temperature raised T_r by the data is Eq.(8).

$$T_r = \frac{f(x, T_{heat})(t_n - t_{n-1})}{C\rho d} \quad (8)$$

where C is specific heat capacity, which is the heat capacity per unit mass of a material. d is the thickness of the object, ρ is density, which is mass per unit volume. For calculating data temperature, C, d and ρ are constant.

Let $\beta = \frac{\sigma r^2}{Cd}$, and $(t_n - t_{n-1})$ is modeled as the distance x between the data and heat source T_r is modelled by Eq.(9).

$$T_r = \beta \frac{{T_{heat}}^4}{\rho x} \quad (9)$$

The data temperature model for `TempSched` includes both temperature decay and warming. $T(t_n)$ is Eq.(10) and (11). c is a discrete function in Eq.(12).

$$T(t_n) = T_f + cT_r \quad (10)$$

$$T(t_n) = T(t_{n-1})e^{-kx} + c\beta \frac{{T_{heat}}^4}{\rho x} \quad (11)$$

$$c = \begin{cases} 1, & \text{When data is accessed at } \mathrm{t_n} \\ 0, & \text{When data is not accessed at } \mathrm{t_n} \end{cases} \quad (12)$$

The ρ of data is considered as a constant here. Let $\gamma = \frac{\beta}{\rho}$, $T(t_n)$ is in Eq.(13).

$$T(t_n) = T(t_{n-1})e^{-kx} + c\gamma \frac{{T_{heat}}^4}{x} \quad (13)$$

$$T_i(t_n) = T(t_{n-1})e^{-k(t_n - t_{n-1})} + \gamma s \frac{{T_{heat}}^4}{(t_n - t_{n-1})} \quad (14)$$

So, we get Eq.(14). If accessed s times in $(t_n - t_{n-1})$, the temperature of $data_i$ is $T_i(t_n)$.

We have solved the limitation that the increased temperature of the existing method is only related to the number of accesses. To intuitively understand the advantages of adding access intervals, we provide two examples in Figure 2.

(a) Accelerate the necessary scheduling. In Figure 2(a), The time series was accessed 5 times between t_0 and t_5. In AdjustDTM's temperature model, the temperature increased is only related to the access times. Although the data continues to heat up and completes two scheduling operations, it is not successfully scheduled to the edge until t_4. In contrast, `TempSched`'s temperature model considers the time intervals. With smaller access time intervals, the temperature increased is significant, and the data keeps heating up. Although `TempSched` also takes two scheduling operations, it successfully schedules to the device by t_3. Thus, `TempSched` completes scheduling faster than AdjustDTM and can assist the device in real-time queries.

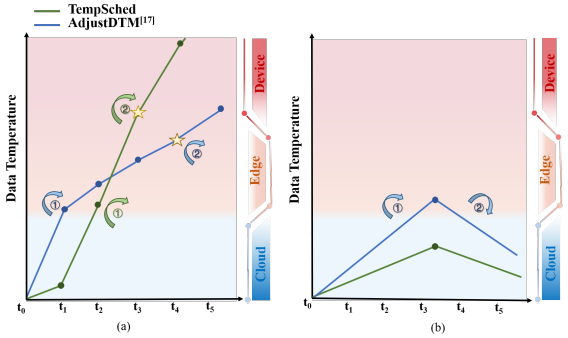


Fig. 2. The simple examples of using `TempSched` in the CED architecture have better performance than AdjustDTM.

(b) Avoid the redundant scheduling. In Figure 2(b), The time series was accessed only once between t_0 and t_5. AdjustDTM's temperature model causes the data to heat up continuously, resulting in one scheduling of hot data to the edge. As the data temperature decreases, the data are scheduled back from the edge to the cloud. In contrast, `TempSched`'s temperature model is related to the time interval. With a large access time interval, the temperature increase is minimal, and no scheduling occurs, avoiding unnecessary scheduling.

Temperature Encapsulation Method. Based on the data temperature model, we can calculate the temperature of time series. However, the determination when to update the temperature and how to store it are also challenging problems. If we update a large amount of time series temperature all the time, it will be a huge overhead on computing resources. Since the time series query is mostly periodic, we can update the temperature periodically. Thus, we update the data temperature within a time window. The updated data temperature algorithm is shown in Algorithm 1.

When data x_i is inserted, its initial temperature is set to be the same as the heat source temperature. We maintain a tuple for x_i to record the timestamp of the last access and the current temperature (Line 4). When the query for x_i arrives, we do not update the temperature of x_i immediately but increment s (Lines 6–8). We update the temperature records of all data uniformly when the time of our specified time window arrives, and set s to zero (Lines 9–12).

The determination of the time window size for updating the temperature is difficult. A large window could lead to inaccurate temperature estimations, while a small window might degrade system performance. Therefore, we proposed an automated setting strategy for the temperature update window w based on a regression model. Firstly, we extracted feature

Algorithm 1: Time series temperature updating

Input: Temperature of the heat source: T_{heat}, Time window: $w = t_n - t_{n-1}$, Times of access: s
Output: Data temperature at t_n: $T_i(t_n)$

insert data x_i;
$T_0 = T_{heat}$;
$s = 0$;
Maintain a tuple ($Timestamp_{query}(t_{n-1})$, $T_i(t_n)$) for each time series;
for *i=0; i < data.num ; i++* **do**
  if *Time interval*$<w$ **then**
    if *query the Data* **then**
      $s = s + 1$;
  else if *Time interval*$= w$ **then**
    $T_i(t_n) = T(t_{n-1})e^{-k(t_n - t_{n-1})} + \gamma s \frac{{T_{heat}}^4}{(t_n - t_{n-1})}$;
    $s = 0$;
    update the tuple ($Timestamp_{query}(t_{n-1})$, $T_i(t_n)$);
return

from historical data and workloads, including the data volume, timestamps, access frequency of the workload and the amount of accessed data. They are used with the temperature update time and accuracy under different temperature update windows to train the linear regression model. The training time of the linear regression model can reducing the model training overhead. For example, in a factory scenario: given a historical dataset, let X_1 is the data volume, X_2 is the data timestamp, X_3 is the execution time of the workload, X_4 is the amount of accessed data, X_5 is the temperature update window, Y_i is the temperature update time and the accuracy of the temperature. The models used to predict the temperature window in Eq.(15).

$$Y_i = \beta_{0_i} + \beta_{1_i} X_1 + \beta_{2_i} X_2 + \beta_{3_i} X_3 + \beta_{4_i} X_4 + \beta_{5_i} X_5 + \epsilon \quad (15)$$

Where β_{0_i} is the intercept, β_{1_i}, β_{2_i}, β_{3_i}, β_{4_i}, β_{5_i} are the parameter, and ϵ is the error term. Based on the prediction results, we can determine the window size. It balanced temperature estimation accuracy and system performance under different workloads. The time complexity of Algorithm 1 is $O(n)$, which depends on the data scale. To ensure scalability, the time series temperature updating method is designed for on-demand updates, occurring only prior to storage scheduling. Additionally, although the volume of cold data on the cloud is substantial, temperature updates only take place for the accessed partitions. Therefore, this temperature update method is scalable.

In the implementation, to facilitate the temperature update and obtain the time series temperature in real time, `TempSched` adds two new fields after the temporal data fields, namely Last Query Timestamp and Time Series Temperature in Table I. They are used to store the current timestamp and the current temperature of the data being accessed. Last Query Timestamp type is TIMESTAMP, which occupies 4 bytes of storage space, and Time Series Temperature type is FLOAT, which occupies 4 bytes, so the increased storage overhead is $8a$ bytes, where a is the amount of temporal data. In order to reduce the storage overhead, the Last Query Timestamp and Time Series Temperature storage of cold data will not be maintained when the data temperature is too cold.

TABLE I
TEMPERATURE STORAGE STRUCTURE

Measurement					
Tags	Timestamp	Field1	...	**Last Query Timestamp**	**Time Series Temperature**

Optimized for time series temperature updates. The time series is massive and updating fast. In order to reduce the data temperature update overhead, we update it on demand. We can update certain data temperature selectively based on the scenario.

C. Time Series Temperature Predictor

The temperature calculator can already get the current temperature of time series. Due to the complexity of access patterns, if changes in access patterns are not promptly reflected in the hot and cold data classification, it will result in a system performance decline. To address this, we introduce a temperature predictor. Based on the previously proposed temperature model, the core function of the temperature predictor is to forecast future access patterns. It is similar to existing research includes workload prediction. However, time series queries often involve batch access based on time periods, so predicting the future access patterns of time series is challenging, which requires not only forecasting future query workloads but also identifying which time series will be accessed.

Key idea. The time series temperature predictor incorporates an integrated workload prediction model that accurately forecasts future access patterns of time series. Specifically, since our temperature calculations focus on individual data points and time series is typically accessed in batches, we designed a frequent timestamp discovery algorithm. This algorithm identifies specific access patterns of data points to calculate their future temperature.

Workload Forecasting Method. To predict the query workload arrival rate, we rely on historical query logs to build a workload forecasting model. Since all queries in query logs will consume too much time in training models, we designed the query template extraction method and the query template clustering method to accelerate the prediction.

(i) **Query Template Extraction.** We extract the template for each input query with the following two steps. First, we record the timestamp of the query, $record = Now() - time_{end}, Now() - time_{start}$. Then we replace the timestamp with a placeholder. Since we only need to predict the frequently accessed data, we can ignore further operations on the data such as aggregation. Therefore, we remove such operations. An example of query template extraction is shown in Figure 3, where the timestamp is replaced with a placeholder, and the AVG operation in the query is removed.

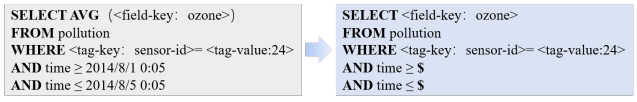


Fig. 3. An Example Query Template Extraction.

(ii) **Query Template Clustering.** `TempSched` utilizes a modified version of the DBSCAN [35] algorithm. The original

DBSCAN algorithm evaluates whether an object belongs to a cluster by examining the minimum distance between the object and any core object in the cluster. However, we want to assign templates to clusters based on how close they are to a cluster's center and not just any random core object, since we use the cluster centers to represent templates belonging to that cluster and establish prediction models based on these centers. Additionally, the online extension of the DBSCAN algorithm incurs a high overhead in updating clusters [36].

Although QB5000 [32] also adopts this clustering approach, it fails to consider cases where query template arrival rate waveforms are similar but not aligned on the time axis. To avoid misjudging similar templates as dissimilar due to misaligned time axes, we propose a query template clustering algorithm based on Dynamic Time Warping (DTW) [37]. We use the DTW distance function as a similarity measure between the historical arrival rate waveforms of two templates. Only when the similarity within each cluster of query templates is high, can the deployment of prediction models yield more accurate arrival rate predictions. To achieve this, we introduce a threshold α to the similarity measure, and only query templates with a similarity less than α will be assigned to a new cluster.

The Query Template Clustering Algorithm is shown in Algorithm 2. For each new template, the distance between its historical arrival rate and the center of any cluster is first calculated, and the template is assigned to the cluster with the smallest distance and less than α. We use a kd-tree to quickly find the closest center of existing clusters to the template in a high-dimensional space [38]. If there are no existing clusters (this is the first query), or if no cluster has a center close enough to the template, `TempSched` will create a new cluster with the template as the only member (Line 1–10). `TempSched` periodically checks the similarity of the previous templates to the centers of the clusters that they belong to. If the similarity of a template is no longer less than α, `TempSched` removes it from the current cluster and repeats Lines 1–10 to find a new cluster (Lines 11–13). If the cluster does not receive a template for a long period of time, `TempSched` will delete the cluster(Lines 14–15). The complexity of these steps is $O(n \log n)$, where n is the number of templates in the workload.

At this stage, in `TempSched`'s time series data temperature predictor, the original query logs have been transformed into templates and grouped into clusters. Specifically, to quickly decide whether to recluster, `TempSched` not only periodically runs the query template clustering algorithm but also continuously monitors changes in access patterns. If the proportion of new access patterns exceeds a preset threshold, the system will recluster to adapt to the updated data access needs. Setting this threshold requires balancing system flexibility and performance overhead. It depends on the system resource utilization and response time requirements. We plan to further investigate optimal threshold setting in future work to enhance adaptability and stability of the system.

The next step is to construct prediction models to forecast the arrival rate patterns for queries within each cluster. It is important to note that even with the use of clustering techniques to reduce the total number of queries for which `TempSched` needs to train models, real-world applications still tend to have a large number of clusters due to the long-tail distribution of arrival rate patterns. Only a few large clusters exhibit major workload patterns, while several small clusters contain noise patterns. These small clusters typically consist of queries that appear only a few times and introduce noise into the model with little benefit, as they do not represent the primary workload of the application. Therefore, `TempSched` does not build models for these small clusters.

Algorithm 2: Query Template Clustering

Input: $templates$ is the new templates set, threshold α
Output: $clusters = \{c_1, c_2, ..., c_m\}$ is the cluster set

1 $templates$ marked as unprocessed ;
2 **for** *i=0; i* $< templates.num$ *; i++* **do**
3 **if** *i=1* **then**
4 Create c_m for $template_i$;
5 **else**
6 Calculate the DTW between $template_i$ and each center of $clusters$;
7 **if** DTW_i *between* $template_i$ *and* $center_m$ *of* $clusters$ *is minimum and* $DTW_i < \alpha$ **then**
8 $c_m \leftarrow template_i$;
9 **else**
10 Create c_m for $template_i$;
11 **if** *the* DTW *between* $template_i$ *and* $center_m \geq \alpha$ **then**
12 Drop $template_i$ from c_m ;
13 Repeat Line3–14 for the $template_i$;
14 **if** C_m *does not receive a template for a long time* **then**
15 Drop the c_m from $clusters$;
16 return

(iii) **Ensemble Forecasting Models Training.** To design the prediction model for query arrival rate, we analyze and compare the four most widely used prediction models including Linear Regression (LR), ARIMA, Holt Winter's (HW), and LSTM. We found that the LR and LSTM outperformed the other models. First, LR can avoid overfitting well when dealing with short-term prediction because the intrinsic relationship of data is simple, so LR short-term prediction is very effective. As for long-term prediction, Long Short Term Memory (LSTM) as a variant of RNN enables the network to automatically learn the periodicity and repetitive trends of data points in time series, which is more suitable for predicting nonlinear scenarios compared to traditional RNN.

Then, how to combine the advantages of multiple models? In other application fields, the effective method of researchers is to use an ensemble model, which combines multiple models for average prediction. In prediction tasks, ensemble methods combine multiple machine learning techniques into a single prediction model to reduce variance or bias (e.g., Boosting) [39]. Previous work has shown it works well and they are often the top winners in data science competitions [40], [41]. To combine the advantages of LR and LSTM, `TempSched` uses an ensemble learning model including them.

The ensemble model combines the prediction results of the LR and LSTM models through the weight coefficient. The weight coefficient is calculated by the minimum sum of squared errors, and the model with higher prediction accuracy is given greater weight. The weights are calculated as shown in Eq.(16).

$$W_i = E_i / \sum_{i=1}^{n} E_i \tag{16}$$

where W_i is the weight of $Model_i$, E_i is sum of squared errors of $Model_i$, n is the type of prediction model, here is LR and LSTM.

The LR model is shown in Eq.(17), where a, b_1, b_2, ... , b_n are regression parameter.

$$y = a + b_1 x_1 + b_2 x_2, ..., b_n x_n + \varepsilon \tag{17}$$

The LSTM model is shown in Eq.(18–23), where W_f, W_g, W_i and W_o represent the weights associated with x_t and h_{t-1}, respectively, b_f, b_g, and b_i represent the bias, and σ represents the activation function.

$$f_t = \sigma(W_{fx} x_t + W_{fh} h_{t-1} + b_f) \tag{18}$$

$$i_t = \sigma(W_{ix} x_t + W_{ih} h_{t-1} + b_i) \tag{19}$$

$$g_t = tanh(W_{gx} x_t + W_{gh} h_{t-1} + b_g) \tag{20}$$

$$O_t = \sigma(W_{ox} x_t + W_{oh} h_{t-1} + b_o) \tag{21}$$

$$C_t = g_t \times i_t + C_{t-1} \times f_t \tag{22}$$

$$h_t = tanh(C_t) \times O_t \tag{23}$$

The scope of the prediction models is defined based on their horizon and interval. The horizon of a model refers to how far into the future that can forecast, with longer horizons generally leading to less accurate predictions. The model's ability to forecast within a specific time granularity is referred to as the prediction interval. For instance, the model may predict the number of queries to be executed each minute or hour. `TempSched` constructs a prediction model for each required prediction horizon.

Frequent Access Timestamp Estimation Method. After the query workload arrival rate is predicted, we know the time series and related fields which will be accessed next, because the query in the temporal data is more concerned about the data in a period in time, i.e., the $record$ records a period in time. Then which timestamp range of these data will be accessed frequently? The core lies in how to count the frequency of the accessed timestamp ranges in the query log. Here, we cut the period time into time points, so the problem is transformed into the problem to count the frequency of accessing the timestamp in the query log, which is a classic problem as follows [42].

Problem Definition. Given timestamp data streams $\sigma = \{a_1, a_2, ..., a_m\}$, where m is the size of the data stream, $a_i \in \{1, 2, ..., n\}$. The number of occurrences of each element is $f = (f_1, f_2, ..., f_n)$, where f_i is the number of occurrences of the i_{th} element. We can easily derive $m = \sum_{i=1}^{n} f_i$. Given the parameter k, if we find all the elements with more occurrences than m/k, it is the output set $\{j \mid f_j > m/k\}$.

We consider that if we maintain a counter for each element, we need to have n counters that can be computed in $O(n)$ time. However, we do not have enough memory to store the whole data stream. Sketches have been used for counting problems in data streams. We chose the Misra-Gries (MG) [42] sketch over other space-saving sketches for three main reasons. Firstly, compared to Count-Min Sketch [43] and Space-Saving Sketch [44], MG can accurately identify frequent items, which helps us precisely find frequently accessed timestamps. Secondly, MG does not use hash functions, so it avoids estimation errors from hash collisions. Finally, the MG algorithm is simple and easy to implement, as it uses the fixed-size counter set and straightforward insertion and count-decrement mechanism. Therefore, we approximate the problem based on the MG algorithm with an error rate of ε. Frequent access timestamp mining is shown in Algorithm 3.

Algorithm 3: Frequent Timestamp Mining

Input: Create an array T consists of (x_i,C_{x_i}), k is size of T , $\sigma = \{a_1, a_2, ..., a_m\}$
Output: array T

```
1  T ← ∅ ;
2  for i=0; i < m; i++ do
3      if a_i ∈ T then
4          (a_i, C_{a_i}) ← (a_i, C_{a_i} + 1);
5      else if a_i ∉ T then
6          if T.size < k - 1 then
7              T ← (a_i, 1)
8          for i=0; i < T.size ; i++ do
9              (a_i, C_{a_i}) ← (a_i, C_{a_i} − 1);
10     if C_{a_i} = 0 then
11         drop (a_i, C_{a_i});
12 return
```

We create an array T of size k. For a_i arriving in the data stream in sequence, if a_i is in the T, plus one to its counter C_{a_i} (Lines 2–4); if a_i is not in the T and the number of elements in the T is less than $k-1$, add the a_i to the T and assign the counter to be one; in other cases, the counters of all elements in T subtract one (Lines 5–9). If the counter value of an element in T is 0, the element is removed from the T (Lines 10–11). When the scan of the data stream is completed, the k elements saved in T are the frequent items.

The time complexity is $O(m \times n)$. The space complexity is $O(\varepsilon^{-1} \log mn)$. For any $query_i$, $\widehat{f_i}$ that is satisfied $f_i - \varepsilon n \leq \widehat{f_i} \leq f_i$ is returned. Therefore, the approximation ratio bound of Algorithm 3 is ε. After obtaining the frequent items in the timestamped data stream, combined with the query arrival rate prediction result, we can preheat dataset within the preheat-size constraints. In addition, to ensure that Algorithm 3 can effectively mine frequently accessed timestamps and maintain good performance as the scale of historical data continues to grow, we can also employ the use of sliding windows.

After identifying the frequently accessed timestamps, we can naturally estimate the number of future accesses to the time series. Consequently, it becomes possible to predict the temperature increase of the time series over a certain period. The predicted temperature increase is shown in Eq.(24). We use the $T_{pre}(t_n)$ combined with the $T_i(t_n)$ to determine whether cloud data should be preemptively scheduled for preheating on the edge side.

$$T_{pre}(t_n) = \gamma s_{pre} \frac{{T_{heat}}^4}{(t_n - t_{n-1})} \quad (24)$$

V. EVALUATION

In this section, we discuss our experimental setup and results. In addition, as a case study, we provide a practical application of temperature-aware storage scheduling for time series across CED, which is the smart factory.

A. Experimental Settings

To evaluate the performance of `TempSched`, we implemented a hot and cold data hierarchical storage scheduler. A device (windows system Intel(R) Core(TM) i7-8565U CPU @ 1.80GHz 1.99 GHz), an edge server (Ubuntu 18.04.6 LTS AMD(R) RyZen 5 3600x 6-core processor x 12), and a cloud server consisting of InfluxDB 1.1.1 [45] are in CED.

B. Datasets and Workloads

Since the query logs are difficult to obtain and handle in practice, query generation is a method to replace query logs. However, the current mainstream query generation [46], [47] is mainly for relational data, which is difficult to be directly applied to the time series. Therefore, researchers try to use synthetic query sets for benchmarking on time series databases. Time series database Benchmark (like YCSB-TS [48], TSBS [49], and TS-Benchmark [50]) are all based on query templates to generate queries. However, the generated queries can only cover a small share of simple cases and cannot meet the needs of our workload prediction requirements. To address these problems, we propose a template-based workload generation method for time series. We attempt to discover the patterns of query arrival rates in existing benchmarks such as Cycles, Stability, Spike, and Chaos. Then, we construct the templates with such arrival rate patterns.

1. **Initial Query Template Construction.** To construct query template, it is essential to consider enough cases of query workload comprehensively. To generate diverse templates, we divide the time series queries into three categories: Conditional Query, Aggregate Query, and Group Query. Based on them, we design five initial query templates.

TABLE II
POLLUTION ATTRIBUTES TRANSFORM TO FORMAT OF INFLUXDB

Ozone	particulate matter	CO_2	SO_2	NO_2	longitude	latitude	timestamp
Field					Tag		Timestamp

We use the pollution public dataset [51], which is a collection of air pollution measurements generated based on the air quality metric (449 sensors) measuring data from August 2014 - October 2014. It has 200,000 data points and 313 MB data. Its attributes are in Table II. We will introduce query templates with the pollution dataset [51] as an example.

Initial Query Template 1. Conditional queries on single time series.

Example 1: When people want to travel, they execute this query to find out the air quality changes at the destination over a period of time. Experts predict the future air quality of a place by analyzing the change of air conditions in the past.

SELECT <field-key: ozone>[,<$field-key$>,<$tag-key$>,...or*] **FROM** pollution **WHERE** <$tag-key$: $sensor-id$>=<$tag-value$: id> **AND** time $\geq$? **AND** time $\leq$?

Initial Query Template 2. Conditional queries on multiple time series by time period.

Example 2: If there are multiple sensors at the place, the query will visit the data of multiple sensors when experts plan to observe the change in air quality in multiple places.

SELECT <field-key: ozone>[,<$field-key$>,<$tag-key$>,...or*] **FROM** pollution **WHERE** <$tag-key$: $sensor-id$>=<$tag-value$: id> **IN** { <$tag-id1$>,... } **AND** time $\geq$? **AND** time $\leq$?

Initial Query Template 3. Conditional queries for time series by period time and certain operators.

Example 3: When an air pollutant indicator exceeds or falls below a threshold value, the query needs to be executed to determine the specific time period and to conduct further analysis and warning. Operator can be $>, \geq, <, \leq and =$.

SELECT <field-key: ozone>[,<$field-key$>,<$tag-key$>,...or*] **FROM** pollution **WHERE** <$tag-key$: $sensor-id$>= <$tag-value$: id> **AND** time $\geq$? **AND** time $\leq$? **AND** <$field-key$: ? > **OPER** ?

Initial Query Template 4. Aggregate queries for time series by period time.

Example 4: People can know the approximate air pollution situation of a place over a period in time where the Operator can be AVG, MAX, MIN.

SELECT OPER? (<field-key: ozone>[,<$field-key$>,<$tag-key$>,...or*]) **FROM** pollution **WHERE** <$tag-key$: $sensor-id$>=<$tag-value$: id> **AND** time $\geq$? **AND** time $\leq$?

Initial Query Template 5. Group queries for time series by period time.

Example 5: People compare the air quality of different places to decide their final destination before traveling.

SELECT OPER?(<field-key: ozone>[,<$field-key$>,<$tag-key$>,...or*]) **FROM** pollution **WHERE** <$tag-key$: $sensor-id$>=<$tag-value$: id> **IN** { <$tag-id1$>,<$tag-id2$>,... } **AND** time $\geq$? **GroupBy**

2. **Arrival Pattern.** To simulate real-world scenarios, we describe four common workload patterns that are popular temporal database applications.

Cycles: The number of queries varies periodically. For example, during the day, 7-9 o'clock and 18-21 o'clock may be the peak time of the query. For the same query template, there may be multiple different cycles.

Stability: The number of queries is stable. When experts want to analyze sensor data, they usually execute queries at specific moments (i.e. every half hour).

Spike: The number of queries shows explosive growth and decay. When the sensor is abnormal, for timely detection and repair of abnormalities may lead to a large number of queries in a short period time.

Chaos: Queries are generated randomly and the arrival rate is irregular.

3. **Query Generation.** When initial query templates and arrival patterns are available, query generation can be performed. Query generation is divided into four main steps: first, constructing a summary of query templates for the dataset, then matching the arrival rate pattern for each template, determining the number of queries generated by a template over time based on the arrival rate pattern, and finally populating the templates to generate a collection of queries.

To simulate queries executed over 10 days on pollution data collected from 449 sensors, we generated queries by setting parameters for four different arrival rate patterns and matching

them with query templates. Consequently, the access to data from the 449 sensors resulted in 449 query arrival rate patterns. The total workload arrival rate, calculated every 5 minutes, is illustrated in figure 4. After extracting templates and clustering

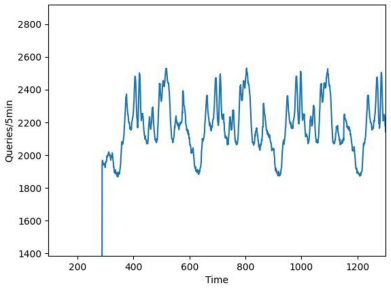


Fig. 4. Workload arrival rate.

the 449 query arrival rate patterns, we trained and predicted using models for the top 3 largest clusters.

C. Time Series Temperature Calculator Evaluation

In this section, we primarily evaluate the ability of `TempSched`'s temperature calculator classifies hot and cold data. The hit rate is used as a metric to assess the impact of data classification on system performance. We compare the temperature calculator, that is, `TempSched` without the temperature prediction (`TempSched`-TP), with three existing methods for classifying hot and cold data.

1. **AdjustDTM.** It modeled the temperature of data based on Newton's law of cooling in [22].
2. **LRU.** The cache replacement strategies represented by Least Recently Used (LRU) in [14].
3. **LFU.** The cache replacement strategies represented by Least frequently Used (LFU) in [15].

All parameter ranges and their default values in the temperature calculator are shown in Table III.

TABLE III
LIST OF PARAMETERS

Parameter	Description	Unit	Value
k	Cooling rate	None	0.01
β	Constant	None	100
ρ	Density	None	0.2
γ	Heating rate	None	500
T_{heat}	Temperature of the heat source	°C	300

We use the *hot database size* to represent the storage space on the edge and device sides, and the *rate of hot database size to CED database size* to represent the proportion of storage space on the edge and device sides relative to the total storage space across the cloud, edge, and device. Figure 5 shows the performance of the four methods with the changeable value of hot database size. It is clear to find that with the increment of cache size, the hit ratio of all four models improved gradually. When the rate of hot database size to CED database size is small, the hit rates of LRU and LFU are low. But from start to end, `TempSched` (-TP) shows the highest cache hit rate. Through the above experiments, `TempSched` (-TP) can achieve about 89% hit rate for data access on the edge and device. It also shows that our proposed temperature model can reduce the storage overhead of CED and improve the efficiency of collaborative queries. We owe it to the successful usage of

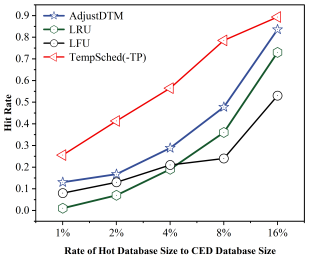


Fig. 5. Time series temperature calculator evaluation.

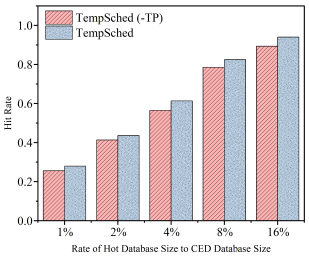


Fig. 6. Time series temperature predictor evaluation.

our data temperature model, which means that leveraging the thermal radiation law to model the access heating of data is more suitable for industrial IoT scenarios.

Summary. In terms of classifying hot and cold data, `TempSched` outperforms state-of-the-art methods. This is because it utilizes Newton's law of cooling and the thermal radiation law, which allows for a quantitative description of the relationship between data temperature increase and access patterns, even under complex time series data access scenarios. Compared to AdjustDTM, `TempSched` achieves over 20% improvement, and compared to LRU and LFU, it demonstrates 30% and 40% improvement, respectively.

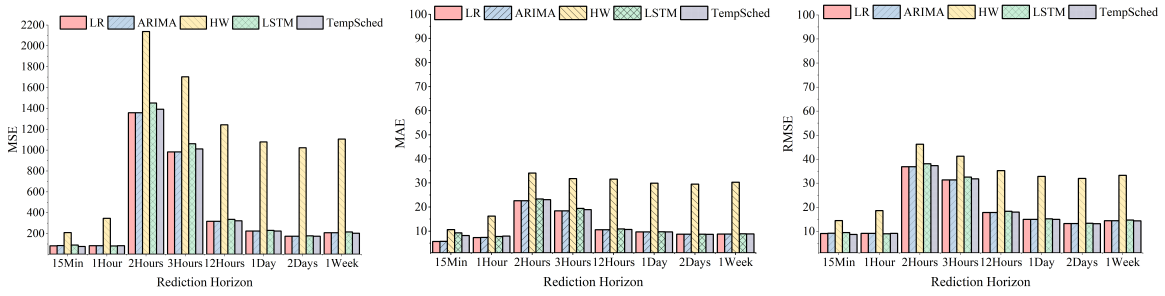
D. Time Series Temperature Predictor Evaluation

In this section, we evaluate the temperature predictor. The hit rate is similarly used as a metric to assess the impact on system performance. We compare `TempSched` to `TempSched` without the temperature prediction (`TempSched-TP`). The experimental results are shown in Figure 6. The temperature predictor can provide approximately 5% improvement in system performance. Such a result confirms that temperature predictor can assist us to identify and preheat data points that seem to be visited frequently shortly. The rows of data with higher temperatures are more likely to be loaded and kept on the edge and device side.

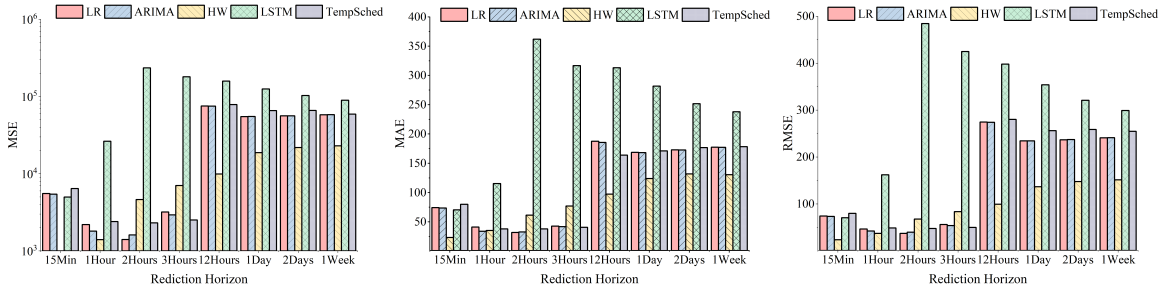
Summary. When the rate of hot database size to CED database size is 1%, the time series temperature predictor can still significantly enhance the efficiency of tiered storage. Because it can forecast the access patterns of time series in future and predict the data temperatures. By anticipating these access patterns, the predictor can adjust the classification of hot and cold data in advance.

E. Workload Forecasting Accuracy Evaluation

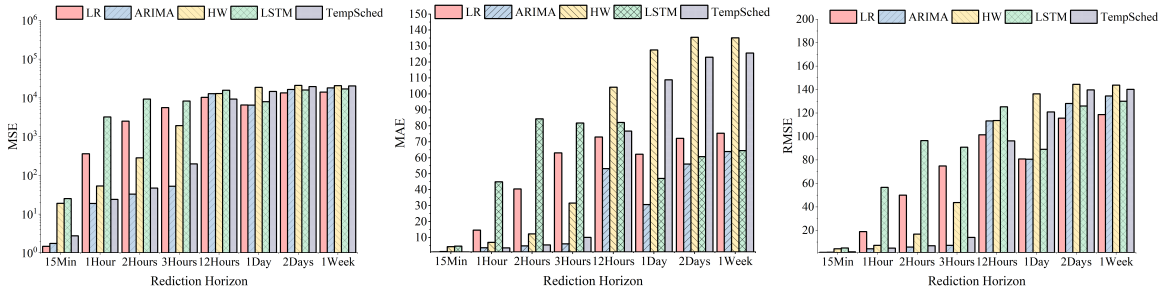
In this section, we evaluate the prediction accuracy of `TempSched`'s prediction models across different prediction horizons. We use Mean Absolute Error (MAE), Mean Square Error (MSE), Root Mean Square Error (RMSE) as evaluation metrics, and set $\alpha = 3.0$ for clustering query template. Next, we deploy prediction models for the top three largest clusters, with Cluster 1 containing 168 query templates, Cluster 2 containing 104 query templates, and Cluster 3 containing 16 query templates. We use 90% of the workload as training data and 10% as testing data. `TempSched` is compared the four most widely used prediction models, namely LR, ARIMA, HW, and LSTM. We take the prediction interval of 5 minutes.



(a) Cluster 1



(b) Cluster 2



(c) Cluster 3

Fig. 7. Workload forecasting model evaluation.

The performance of each model is recorded for eight different prediction horizons in Figure 7.

On Cluster 1, `TempSched`'s prediction accuracy exceeds that of ARIMA, HW, and LSTM across all 8 prediction horizons. Within a prediction horizon of one hour, `TempSched`'s prediction accuracy surpasses LR.

On Cluster 2, `TempSched`'s prediction accuracy exceeds LSTM for prediction horizons greater than 1 hour and less than 12 hours, and after a prediction horizon of 15 minutes, its accuracy matches that of LR and ARIMA.

On Cluster 3, `TempSched`'s prediction accuracy outperforms LR, HW, and LSTM in greater than 15 min and less than 12 hours.

Summary. For shorter horizons, the performance of the LR model is better than more complex learning models. Because of the relationship between the arrival rates observed in the recent past and those expected in the near future tends to be more linear with shorter horizons. In contrast, complex models often overfit the noise in the training data for shorter horizons, resulting in less accurate outcomes. However, as the horizon expands, the relationship between past and future becomes increasingly intricate, favoring models adept at learning complex relationships. In addition, the MSE on Cluster 1 is lower than on Clusters 2 and 3 as Cluster 1 has the most periodic workload pattern. It shows that `TempSched` performs best for workloads with strong cycles. In future work, we will optimize the model to handle more patterns. Overall, `TempSched` is suited for scenarios with a large number of query templates and prediction horizons of approximately one day.

F. Computation Efficiency Evaluation

In this section, we investigate the computational cost of the prediction models. The results are shown in Table IV. As illustrated, simple models have shorter training times. For instance, the training time for LR is 0.2 seconds. Machine learning-based models generally require longer training times. However, these training costs are acceptable. We found that the training time of `TempSched` is lower than that of the LSTM model. This indicates that `TempSched` has advantages in both performance and efficiency, making it suitable for practical applications.

To improve training efficiency, we recommend stopping the training process as soon as the model begins to converge. This can help avoid unnecessary computations and reduce

TABLE IV
COMPUTATION EFFICIENCY.

Model	CPU Time	Inference
LR	0.2s	3ms
ARIMA	20.6s	9ms
HW	4.1s	5ms
LSTM	163.2s	21ms
TempSched	161.9s	20ms

training time. To ensure that our predictive model maintains its advantages without sacrificing system efficiency, we can consider the following strategies to alleviate training overhead:

Incremental Learning: We can adopt an incremental learning strategy. This strategy allows the model to update gradually as it receiving new data, without the need to retrain from scratch. This can reduce training time.

Knowledge Distillation: We can transfer the knowledge from a large model to a smaller model through knowledge distillation. This can significantly lower computational overhead while maintaining high predictive performance.

By combining these strategies, we can reduce the training overhead of TempSched. At the same time, we can ensure its effectiveness in practical applications. This will help us optimize resource usage under different workload conditions, achieving more efficient data management and scheduling.

Summary. Although the training and inference times for TempSched are longer compared to simpler models such as logistic regression (LR), these simple models are only suitable for short-term predictions. For longer horizons, the computational overhead of TempSched is less than that of LSTM. Additionally, we use strategies such as incremental learning to further reduce training overhead.

G. Sensitivity Analysis of k

We now analyze the query hit rates for different values of k in Algorithm 3. As discussed in the section IV, k represents the size of the candidate set in the frequent timestamp mining algorithm, determining the number of frequent timestamps. In these experiments, we compared the query hit rates under different rate of hot database size with k values ranging from 8 to 14.

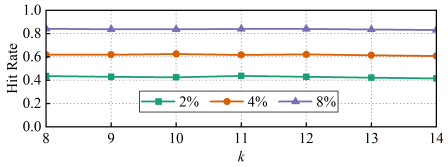


Fig. 8. The hit rate between the three rate of hot database size to CED database size with different k.

Figure 8 shows the query hit rates across different rate of hot database size to CED database size for increasing values of k. The results show that the query hit rates remain stable across different k values.

Summary. The k has a minimal impact on the query hit rate. In other validation experiments, for practical application scenarios, we chose $k = 12$ representing the amount of time series collected over one hour.

H. Case Study

Recall that we mentioned a demand for the hierarchical storage in new CED architecture motivating this work. As a case study, we use a smart factory with CED. In this architecture, the cloud server stores all workshop time series, edge servers store time series for each workshop, and the device stores time series for each device. Since storage on devices and workshop servers is limited, they cannot store all time series, so time series is migrated to the cloud. However, storing all time series in the cloud is impractical because accessing it from the edge requires data transfer, leading to high network costs. Therefore, temperature-aware time series scheduling is essential for collaborative storage in smart factories. Figure 9 shows an example of a smart factory. TempSched can schedule time series across CED:

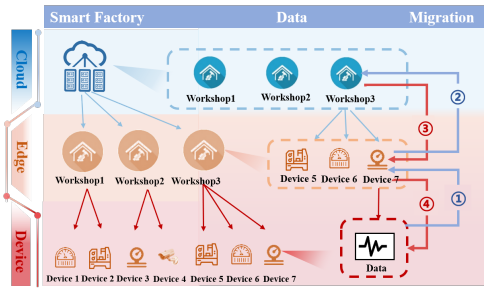


Fig. 9. An example of a smart factory with CED architecture. TempSched can schedule time series across CED.

① Device→Edge When the storage space of a device is full, TempSched sorts the data by temperature and migrates the colder data to the edge.

② Edge→Cloud When the storage space on the edge is full, TempSched sorts the data by temperature and migrates the cold data to the cloud.

③ Cloud→Edge When the edge needs to query data from the cloud, traditional methods transfer this data only when requested. However,TempSched uses a temperature prediction module to forecast data temperature and preemptively migrates the data to the edge.

④ Edge→Device When a device needs to query data from the cloud, traditional methods transfer the data from the cloud to the device only upon request. However,TempSched uses a temperature prediction module to forecast data temperature and preemptively migrates the data to the device.

VI. CONCLUSION

We propose TempSched, a temperature-aware storage scheduler for time series across CED, which can identify hot and cold time series and predict data temperature efficiently to perform storage scheduling in advance. The experimental results show that TempSched can effectively help time series across CED to select optimal storage location. We plan to optimize the workload prediction model and further compress cold data in future work.

ACKNOWLEDGMENT

This work is supported by National Natural Science Foundation of China (NSFC) (62232005, 62202126) and CCF-Huawei Populus euphratica Innovation Research Funding (CCF-HuaweiDB2022004).

REFERENCES

[1] Razvan Gabriel Cirstea, Tung Kieu, Chenjuan Guo, Bin Yang, and Sinno Jialin Pan, "Enhancenet: Plugin neural networks for enhancing correlated time series forecasting," *ICDE*, pp. 1739–1750, 2021.

[2] Zijue Li, Xiaoou Ding and Hongzhi Wang, "An Effective Constraint-Based Anomaly Detection Approach on Multivariate Time Series," *APWeb-WAIM*, pp. 61–69, 2020.

[3] Xiaoou Ding, Yichen Song, Hongzhi Wang, Donghua Yang, Chen Wang and Jianmin Wang, "Clean4TSDB: A Data Cleaning Tool for Time Series Databases," *VLDB*, pp. 4377–4380, 2024.

[4] Cui SS, Wu X, Wang HZ, Wu H, "Multi-modal Data Modeling Technology and Its application for Cloud-edge-device Collaboration. " *Ruan Jian Xue Bao/Journal of Software (in Chinese)*, pp. 1154-1172, 2024.

[5] Spiros Papadimitriou and Philip Yu, "Optimal multi-scale patterns in time series streams," *SIGMOD*, pp. 647–658, 2016.

[6] Xiaoou Ding, Hongzhi Wang, Yitong Gao, Jianzhong Li and Hong Gao, "Determining the currency of dynamic data," *TUR-C*, pp. 1-6, 2017.

[7] Hossain M S, Muhammad G, "Cloud-assisted Industrial Internet of Things (IIoT) – Enabled framework for health monitoring," *Computer Networks*, pp. 192-202, 2016.

[8] Liu H, Han F, Zhang Y, "Malicious traffic detection for cloud-edge-end networks: A deep learning approach," *Computer communications*, pp. 150-156, 2024.

[9] Xiaoou Ding, Yingze Li, Chen Wang, Yida Liu and Jianmin Wang, "TSDDISCOVER: Discovering Data Dependency for Time Series Data," *ICDE,* pp. 3668–3681, 2024.

[10] Faezipour M , Nourani M , Saeed A ,et al, "Progress and challenges in intelligent vehicle area networks," *Communications of the Acm*, pp. 90-100, 2012.

[11] Wang S , Wan J , Zhang D ,et al, "Towards Smart Factory for Industry 4.0: A Self-organized Multi-agent System with Big Data Based Feedback and Coordination," *Computer Networks*, pp. 158-168, 2016.

[12] Alexiou K , Kossmann D , Larson P, "Adaptive range filters for cold data: avoiding trips to Siberia," *Proceedings of the VLDB Endowment*, pp. 1714-1725, 2013.

[13] Xinle Wu, Dalin Zhang, Chenjuan Guo, Chaoyang He, Bin Yang, and Christian S Jensen, "AutoCTS: Automated correlated time series forecasting," *VLDB*, pp. 1-15, 2021.

[14] Dan A, Towsley D, "An approximate analysis of the LRU and FIFO buffer replacement schemes," *Acm Sigmetrics Performance Evaluation Review*, pp. 143-152, 1990.

[15] Robinson J T, Devarakonda M V, "Data cache management using frequency-based replacement," *Acm Sigmetrics Performance Evaluation Review*, pp. 134-142, 1990.

[16] Pathak A, Gurajada A, Khadilkar P, "Life cycle of transactional data in in-memory databases," *2018 IEEE 34th International Conference on Data Engineering Workshops (ICDEW). IEEE*, pp. 122-133, 2018.

[17] Hsieh J W, Kuo T W, Chang L P, "Efficient identification of hot data for flash memory storage systems," *ACM Transactions on Storage*, pp. 22-40, 2006.

[18] HPark D, Du D H C, "Hot data identification for flash-based storage systems using multiple bloom filters," *2011 IEEE 27th Symposium on Mass Storage Systems and Technologies (MSST). IEEE*, pp. 1-11, 2011.

[19] Levandoski J J, Larson P Å, Stoica R, "Identifying hot and cold data in main-memory databases," *2013 IEEE 29th International Conference on Data Engineering (ICDE). IEEE*, pp. 26-37, 2013.

[20] Yulin Xie, "TITLE:Research on Recognition Mechanismof Hot and Cold Data Based on Data Temperature," *Zhejiang University*, 2019.

[21] Jiaxin Xu, "The Research and Implementation of Storage Mechanism for Hot and Cold Data," *University of Electronic Science and Technology*, 2021.

[22] Y. Wang, J. Zhao, Q. Zhou, X. Xiong, H. Zhang and C. Chen, "Identification of Hot data and Caching strategy for Industrial Big Data Based on Temperature Model," *2023 IEEE International Conference on Big Data and Smart Computing (BigComp), Jeju, Korea, Republic of*, pp. 289-290, 2023.

[23] Pelkonen T, Franklin S, Teller J, et al, "Gorilla: A fast, scalable, in-memory time series database," *Proceedings of the VLDB Endowment*, pp. 1816-1827, 2015.

[24] Philippe Bonnet, Johannes Gehrke, Praveen Seshadri, "Towards Sensor Database Systems," *Mobile Data Management*, pp. 3-14, 2001.

[25] Ousmane D, Joel J. P. C. R, Mbaye S, "Real-time data management on wireless sensor networks: A survey," *J. Netw. Comput. Appl*, pp. 1013-1021, 2012.

[26] Akimitsu Kanzaki, Takahiro Hara, Yoshimasa Ishi, Tomoki Yoshihisa, Yuuichi Teranishi, Shinji Shimojo, "X-Sensor: Wireless Sensor Network Testbed Integrating Multiple Networks," *Wireless SensorNetwork Technologies for the Information Explosion Era*, pp. 249-271, 2010.

[27] Fei Hu, Xiaojun Cao, "Sensor Data Management," *SCRC Press*, pp. 237–257, 2010.

[28] Srikrishna Prasad, Avinash Shankaranarayanan, "Smart meter data analytics using OpenTSDB and Hadoop," *ISGT Asia*, pp. 1-6, 2013.

[29] https://iotdb.apache.org/

[30] https://taosdata.com/cn/

[31] Holze M, Ritter N, "Towards workload shift detection and prediction for autonomic databases," *Proceedings of the ACM Ph.D.workshop in CIKM*, pp. 109-116, 2007.

[32] Ma L, Aken D V, Hefny A, et al, "Query-based Workload Forecasting for Self-Driving Database Management Systems," *the International Conference*, 2018.

[33] Liu C, Mao W, Gao Y, et al, "Adaptive recollected RNN for workload forecasting in database-as-a-service," *International Conference on Service-Oriented Computing. Springer, Cham*, pp. 431-438, 2020.

[34] Huang X, Cheng Y, Gao X, et al, "TEALED: A Multi-Step Workload Forecasting Approach Using Time-Sensitive EMD and Auto LSTM Encoder-Decoder," *International Conference on Database Systems for Advanced Applications. Springer, Cham*, pp. 706-713, 2022.

[35] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," *In KDD*, pp. 226–231, 1996.

[36] M. Ester, H.-P. Kriegel, J. Sander, M. Wimmer, and X. Xu, " Incremental clustering for mining in a data warehousing environment," *In VLDB, volume 98*, pp. 323–333, 1998.

[37] Rakthanmanon T, Campana B, Mueen A, et al, "Searching and Mining Trillions of Time Series Subsequences under Dynamic Time Warping," *SIGKDD explorations*, 2012.

[38] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, pp. 509–517, 1975.

[39] D. W. Opitz and R. Maclin, "Popular ensemble methods: An empirical study," 1999.

[40] R. Polikar, "Ensemble based systems in decision making," *IEEE Circuits and systems magazine*, pp. 21–45, 2006.

[41] Z.-H. Zhou, "Ensemble methods: foundations and algorithms," *CRC Press*, 2012.

[42] Misra J, Gries D, "Finding repeated elements. Science of Computer Programming," pp. 143–152, 1982.

[43] Cormode G , Muthukrishnan S, "An Improved Data Stream Summary: The Count-Min Sketch and Its Applications," pp. 58–75, 2004.

[44] Metwally A , Agrawal D , El Abbadi A, "Efficient Computation of Frequent and Top-k Elements in Data Streams" pp. 1–12, 2005.

[45] https://docs.influxdata.com/influxdb/v1.7/concepts/insights_tradeoffs/, 2021.

[46] T. P. P. Council, "Tpc benchmark h (decision support)," *Standard Specification, Revision*, 1(0), 1999.

[47] M. Poess and C. Floyd, "New tpc benchmarks for decision support and web commerce," *ACM Sigmod Record*, pp. 64–71, 2000.

[48] YCSB-TS. http://tsdbbench.github.io/YCSB-TS, 2010.

[49] timescale. Time series benchmark suite. https://github.com/timescale/tsbs, 2020.

[50] Y. Hao et al, "TS-Benchmark: A Benchmark for Time Series Databases," *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pp. 588–599, 2021.

[51] Pollution data. http://iot.ee.surrey.ac.uk:8080/datasets.html, 2014.